UNIVERSIDAD NACIONAL DEL ALTIPLANO
Escuela Profesional de Ingeniería de Sistemas

GUÍA DE PRÁCTICA DE LABORATORIO

SEMANA 4

Interfaces, Clases Abstractas y Composición frente a Herencia

Semestre 2026-II · Principio de Inversión de Dependencias (DIP) — anticipo S9

Curso:	Programación Orientada a Objetos II — SIS206
Semana:	4 (Interfaces, Abstracción y Composición)
Lenguaje:	Python 3.12+ / C++17
Duración:	2 horas de práctica (laboratorio)
Docente:	Dr. Aldo Hernán Zanabria Gálvez
ORCID / RENACYT:	0000-0003-3314-8768 | P0020608

Puno — Perú — 2026

I. OBJETIVO GENERAL

Diseñar soluciones orientadas a objetos utilizando clases abstractas como contratos de interfaz, composición como alternativa preferida a la herencia de implementación, y el principio de dependencia de abstracciones sobre concreciones — anticipando formalmente el DIP de SOLID que se estudiará en la Semana 9.

1.1 Objetivos Específicos

- Distinguir entre clase abstracta (ABC en Python / clase con métodos virtuales puros en C++) e interfaz (protocolo en Python / clase abstracta sin datos en C++).
- Aplicar composición de objetos para construir comportamientos complejos sin acoplarse a implementaciones específicas.
- Diseñar código que depende de abstracciones (interfaces/ABCs) y no de clases concretas.
- Refactorizar el Proyecto Integrador reemplazando herencias no justificadas por composición.

II. MARCO TEÓRICO

2.1 Clase Abstracta vs Interfaz

Una clase abstracta define un contrato parcial: puede tener atributos y algunos métodos implementados, pero obliga a las subclases a completar los métodos abstractos. Una interfaz pura define solo el contrato (firmas de métodos) sin ninguna implementación ni estado:

```
# Python — clase abstracta con ABC (estado + contrato)
```

```python
from abc import ABC, abstractmethod

class Notificable(ABC):
    """Interfaz pura — sin estado, solo contrato."""

    @abstractmethod
    def enviar(self, destinatario: str, mensaje: str) -> bool: ...

    @abstractmethod
    def es_disponible(self) -> bool: ...


class ServicioNotificacion(ABC):
    """Clase abstracta — tiene estado compartido + métodos abstractos."""

    def __init__(self, nombre: str):
        self._nombre = nombre
        self._enviados = 0

    @abstractmethod
    def _enviar_impl(self, dest: str, msg: str) -> bool: ...

    def enviar(self, dest: str, msg: str) -> bool:
        """Template method — implementa el flujo, delega el detalle."""
        if not self.es_disponible():
            return False
        exito = self._enviar_impl(dest, msg)
        if exito:
            self._enviados += 1
        return exito

    @property
    def total_enviados(self) -> int:
        return self._enviados
```

```cpp
// C++17 — clase abstracta pura (interfaz) vs clase abstracta con estado

// Interfaz pura — sin datos, sin implementaciones
class INotificable {
public:
    virtual ~INotificable() = default;
    virtual bool enviar(const std::string& dest,
                        const std::string& msg) = 0;
    virtual bool esDisponible() const = 0;
};

// Clase abstracta — con datos compartidos + Template Method
class ServicioNotificacion : public INotificable {
protected:
    std::string nombre_;
    int enviados_ = 0;

    virtual bool enviarImpl(const std::string& dest,
                            const std::string& msg) = 0;
public:
    explicit ServicioNotificacion(std::string nombre)
```

```
        : nombre_(std::move(nombre)) {}

    bool enviar(const std::string& dest,
                const std::string& msg) override {
        if (!esDisponible()) return false;
        bool ok = enviarImpl(dest, msg);
        if (ok) ++enviados_;
        return ok;
    }
    int totalEnviados() const { return enviados_; }
};
```

2.2 Composición sobre Herencia

El principio de diseño "Favorecer la composición sobre la herencia" (GoF, 1994) establece que los comportamientos deben ensamblarse combinando objetos independientes en lugar de crear jerarquías profundas de herencia. La composición produce código más flexible y fácil de cambiar:

```
# ── HERENCIA (frágil) ──────────────────────────────────────────
class Reporte:
    def generar(self, datos): ...

class ReporteCSV(Reporte):
    def generar(self, datos): ...

class ReportePDF(Reporte):
    def generar(self, datos): ...

class ReporteCSVConFirma(ReporteCSV):     # jerarquía frágil
    def generar(self, datos): ...


# ── COMPOSICIÓN (flexible) ─────────────────────────────────────
class Exportador(ABC):
    @abstractmethod
    def exportar(self, contenido: str, ruta: str) -> None: ...

class Firmador(ABC):
    @abstractmethod
    def firmar(self, ruta: str) -> None: ...


class Reporte:                              # depende de abstracciones
    def __init__(self, exportador: Exportador,
                 firmador: Firmador | None = None):
        self._exportador = exportador
        self._firmador   = firmador

    def generar(self, datos: dict, ruta: str) -> None:
        contenido = self._formatear(datos)
        self._exportador.exportar(contenido, ruta)
        if self._firmador:
            self._firmador.firmar(ruta)

    def _formatear(self, datos: dict) -> str:
        return str(datos)    # placeholder
```

```
# Comportamientos concretos — intercambiables sin tocar Reporte
class ExportadorCSV(Exportador):
    def exportar(self, contenido, ruta): ...

class ExportadorPDF(Exportador):
    def exportar(self, contenido, ruta): ...

class FirmadorDigital(Firmador):
    def firmar(self, ruta): ...
```

2.3 Inyección de Dependencias (anticipo DIP)

La Inversión de Dependencias (DIP — "D" de SOLID, Semana 9) establece que los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones. La composición con interfaces es el mecanismo que lo hace posible. Cuando un objeto recibe sus dependencias desde el exterior (en lugar de crearlas internamente) se llama Inyección de Dependencias:

```
# Sin DI — Reporte CREA su exportador (dependencia rígida)
class ReporteRigido:
    def __init__(self):
        self._exportador = ExportadorCSV()  # difícil de cambiar, difícil de probar


# Con DI — Reporte RECIBE su exportador (dependencia flexible)
class Reporte:
    def __init__(self, exportador: Exportador):  # recibe la abstracción
        self._exportador = exportador

# En el punto de composición (main/factory) se decide la implementación concreta
reporte_csv = Reporte(ExportadorCSV())
reporte_pdf = Reporte(ExportadorPDF())

# Para pruebas unitarias — basta un exportador mock
class ExportadorMock(Exportador):
    def __init__(self):
        self.llamadas = []
    def exportar(self, contenido, ruta):
        self.llamadas.append((contenido, ruta))

mock = ExportadorMock()
reporte = Reporte(mock)
reporte.generar({"datos": [1,2,3]}, "test.csv")
assert len(mock.llamadas) == 1  # verificable sin sistema de archivos real
```



Conexión con Patrones de Diseño

La composición con interfaces es la base de la mayoría de los patrones GoF: Strategy (composición de algoritmos), Decorator (composición de comportamientos), Observer (composición de oyentes) y Factory Method (composición de objetos). La Semana 10-12 formalizará estos patrones; hoy construye la intuición de diseño que los sustenta.

III. MATERIALES

Herramienta	Uso

Python 3.12+ / Colab	ABC, Protocol, composición e inyección de dependencias
g++ -std=c++17 -Wall	Clases abstractas puras, herencia múltiple de interfaz
Draw.io / StarUML	Diagrama refactorizado del Proyecto Integrador

IV. INSTRUCCIONES PREVIAS

1. Abrir el notebook de Colab y agregar una nueva sección "Semana 4 — Interfaces y Composición".
2. Tener disponible el código de herencia de la Semana 3 — esta práctica refactoriza parte de ese trabajo.
3. Compilar con: g++ -std=c++17 -Wall -Wextra -o prog main.cpp
4. Commit al finalizar: "feat(semana4): interfaces y composicion sobre herencia"

V. DESARROLLO DE ACTIVIDADES

ACTIVIDAD 1 — Diseñar interfaces con ABC y Protocol (Python) — Python + C++

Implemente el sistema de notificaciones del marco teórico. Defina las interfaces como ABCs y proporcione dos implementaciones concretas: NotificadorEmail y NotificadorSMS. Luego implemente un SistemadeAlertas que use composición con una lista de notificadores.

```python
# ── Práctica S4 — Actividad 1: Sistema de Notificaciones ──────────────
from abc import ABC, abstractmethod
from typing import Protocol, runtime_checkable


# Interfaz pura con ABC
class INotificador(ABC):
    @abstractmethod
    def enviar(self, destinatario: str, mensaje: str) -> bool: ...

    @abstractmethod
    def es_disponible(self) -> bool: ...

    @property
    @abstractmethod
    def nombre(self) -> str: ...


# Interfaz con Protocol (duck typing estructural — sin herencia requerida)
@runtime_checkable
class ProtocolNotificador(Protocol):
    def enviar(self, destinatario: str, mensaje: str) -> bool: ...
    def es_disponible(self) -> bool: ...


class NotificadorEmail(INotificador):
    def __init__(self, servidor_smtp: str, puerto: int = 587):
        self._servidor = servidor_smtp
        self._puerto   = puerto
        self._activo   = True

    @property
```

```
    def nombre(self) -> str: return f"Email ({self._servidor})"

    def es_disponible(self) -> bool: return self._activo

    def enviar(self, dest: str, msg: str) -> bool:
        # TODO: simular envío (imprimir el mensaje, retornar True)
        print(f"[EMAIL → {dest}]: {msg[:50]}...")
        return True


class NotificadorSMS(INotificador):
    MAX_CHARS = 160

    def __init__(self, proveedor: str):
        self._proveedor = proveedor
        self._activo = True

    @property
    def nombre(self) -> str: return f"SMS ({self._proveedor})"

    def es_disponible(self) -> bool: return self._activo

    def enviar(self, dest: str, msg: str) -> bool:
        if len(msg) > self.MAX_CHARS:
            raise ValueError(f"SMS excede {self.MAX_CHARS} caracteres")
        print(f"[SMS → {dest}]: {msg}")
        return True


class SistemaAlertas:
    """Usa composición — no sabe qué tipo de notificador usa."""

    def __init__(self, notificadores: list[INotificador]):
        if not notificadores:
            raise ValueError("Se requiere al menos un notificador")
        self._notificadores = notificadores

    def agregar_notificador(self, n: INotificador) -> None:
        self._notificadores.append(n)

    def alertar(self, dest: str, mensaje: str) -> int:
        """Envía la alerta por TODOS los canales disponibles."""
        enviados = 0
        for n in self._notificadores:
            if n.es_disponible():
                try:
                    if n.enviar(dest, mensaje):
                        enviados += 1
                except Exception as e:
                    print(f"Error en {n.nombre}: {e}")
        return enviados


# ── Pruebas ──────────────────────────────────────────────────────
sistema = SistemaAlertas([
    NotificadorEmail("smtp.unap.edu.pe"),
    NotificadorSMS("Movistar"),
```

```
])

enviados = sistema.alertar(
    "jmamani@unap.edu.pe",
    "ALERTA: Temperatura del Lago Titicaca supera 18°C"
)
print(f"\nAlerta enviada por {enviados} canales.")

# Verificar que funciona con cualquier INotificador (LSP + composición)
assert isinstance(sistema._notificadores[0], INotificador)
assert isinstance(sistema._notificadores[0], ProtocolNotificador)
```

ACTIVIDAD 2 — Composición sobre herencia — Sistema de reportes

Python + C++

Implemente el sistema de reportes del marco teórico aplicando composición. Luego compare este diseño con uno basado en herencia y responda las preguntas de análisis.

```
# ── Práctica S4 — Actividad 2: Sistema de Reportes con Composición ──────
from abc import ABC, abstractmethod
from pathlib import Path


class Exportador(ABC):
    @abstractmethod
    def exportar(self, contenido: str, ruta: str) -> None: ...

    @property
    @abstractmethod
    def extension(self) -> str: ...


class Firmador(ABC):
    @abstractmethod
    def firmar(self, ruta: str) -> str:
        """Retorna ruta del archivo firmado."""
        ...


class ExportadorCSV(Exportador):
    @property
    def extension(self) -> str: return ".csv"

    def exportar(self, contenido: str, ruta: str) -> None:
        print(f"[CSV] Exportando a {ruta}{self.extension}")
        # En un sistema real: escribir el archivo


class ExportadorJSON(Exportador):
    @property
    def extension(self) -> str: return ".json"

    def exportar(self, contenido: str, ruta: str) -> None:
        print(f"[JSON] Exportando a {ruta}{self.extension}")
```

```
class FirmadorDigital(Firmador):
    def __init__(self, clave: str):
        self._clave = clave

    def firmar(self, ruta: str) -> str:
        ruta_firmada = ruta + ".signed"
        print(f"[FIRMA] Firmando {ruta} → {ruta_firmada}")
        return ruta_firmada


class GeneradorReporte:
    """Depende de abstracciones — no de implementaciones concretas."""

    def __init__(self, exportador: Exportador,
                 firmador: Firmador | None = None):
        self._exportador = exportador
        self._firmador   = firmador

    # ---- inyección de dependencia en tiempo de ejecución ---------------
    def cambiar_exportador(self, exp: Exportador) -> None:
        self._exportador = exp

    def generar(self, datos: dict, nombre: str) -> str:
        contenido = self._formatear(datos)
        ruta_base = f"reportes/{nombre}"
        self._exportador.exportar(contenido, ruta_base)
        ruta_final = ruta_base + self._exportador.extension
        if self._firmador:
            ruta_final = self._firmador.firmar(ruta_final)
        return ruta_final

    def _formatear(self, datos: dict) -> str:
        return ", ".join(f"{k}={v}" for k, v in datos.items())


# ── Demostración ──────────────────────────────────────────────────────
datos_planilla = {"total_personal": 45, "monto_total": 185000.50}

# Reporte CSV sin firma
gen = GeneradorReporte(ExportadorCSV())
ruta = gen.generar(datos_planilla, "planilla_agosto")
print(f"Generado: {ruta}")

# Cambiar exportador en tiempo de ejecución — sin tocar GeneradorReporte
gen.cambiar_exportador(ExportadorJSON())
ruta = gen.generar(datos_planilla, "planilla_agosto")
print(f"Generado: {ruta}")

# Reporte CSV con firma digital
gen_firmado = GeneradorReporte(ExportadorCSV(), FirmadorDigital("clave_unap_2026"))
ruta = gen_firmado.generar(datos_planilla, "planilla_firmada")
print(f"Generado y firmado: {ruta}")
```

Preguntas de análisis:

Pregunta	Su respuesta
¿Cuántas clases nuevas necesita si agrega un ExportadorXML al diseño de composición? ¿Y al de herencia?	
¿Cómo probaría GeneradorReporte sin escribir archivos reales? Describa el mock necesario.	
¿Puede combinar ExportadorCSV con FirmadorDigital Y ExportadorJSON sin firma en el mismo programa? ¿Cómo?	

ACTIVIDAD 3 — Interfaces en C++17 — clases abstractas puras

Python + C++

Implemente el sistema de notificaciones en C++17. Defina la clase abstracta pura INotificador y dos implementaciones concretas. Demuestre polimorfismo con vector<unique_ptr> y el sistema de alertas por composición.

```cpp
// ── notificador.hpp — Práctica S4, Actividad 3 ──────────────────────
#pragma once
#include <string>
#include <vector>
#include <memory>
#include <stdexcept>
#include <iostream>

// ── Interfaz pura ────────────────────────────────────────────────────
class INotificador {
public:
    virtual ~INotificador() = default;
    virtual bool enviar(const std::string& dest,
                        const std::string& msg) = 0;
    virtual bool       esDisponible()    const = 0;
    virtual std::string nombre()         const = 0;
};


// ── Implementaciones concretas ──────────────────────────────────────
class NotificadorEmail : public INotificador {
    std::string servidor_;
    bool        activo_ = true;
public:
    explicit NotificadorEmail(std::string servidor)
        : servidor_(std::move(servidor)) {}

    bool esDisponible() const override { return activo_; }

    std::string nombre() const override {
        return "Email (" + servidor_ + ")";
    }

    bool enviar(const std::string& dest,
                const std::string& msg) override {
        std::cout << "[EMAIL → " << dest << "]: "
```

```
                  << msg.substr(0, 50) << "...\n";
        return true;
    }
};


class NotificadorSMS : public INotificador {
    std::string proveedor_;
    bool        activo_ = true;
    static constexpr int MAX_CHARS = 160;
public:
    explicit NotificadorSMS(std::string proveedor)
        : proveedor_(std::move(proveedor)) {}

    bool esDisponible() const override { return activo_; }
    std::string nombre() const override { return "SMS (" + proveedor_ + ")"; }

    bool enviar(const std::string& dest,
                const std::string& msg) override {
        if (static_cast<int>(msg.size()) > MAX_CHARS)
            throw std::invalid_argument("SMS excede 160 caracteres");
        std::cout << "[SMS → " << dest << "]: " << msg << "\n";
        return true;
    }
};


// ── Sistema de Alertas por composición ──────────────────────────
class SistemaAlertas {
    std::vector<std::unique_ptr<INotificador>> notificadores_;
public:
    void agregar(std::unique_ptr<INotificador> n) {
        notificadores_.push_back(std::move(n));
    }

    int alertar(const std::string& dest, const std::string& msg) {
        int enviados = 0;
        for (auto& n : notificadores_) {
            if (!n->esDisponible()) continue;
            try {
                if (n->enviar(dest, msg)) ++enviados;
            } catch (const std::exception& e) {
                std::cerr << "Error en " << n->nombre()
                          << ": " << e.what() << "\n";
            }
        }
        return enviados;
    }
};


// main.cpp
int main() {
    SistemaAlertas sistema;
    sistema.agregar(std::make_unique<NotificadorEmail>("smtp.unap.edu.pe"));
    sistema.agregar(std::make_unique<NotificadorSMS>("Movistar"));
```

```
    int n = sistema.alertar(
        "jmamani@unap.edu.pe",
        "ALERTA: Temperatura Titicaca supera 18 grados"
    );
    std::cout << "Alerta enviada por " << n << " canal(es).\n";
    return 0;
}
```

ACTIVIDAD 4 — Tabla comparativa — ABC Python vs interfaz C++ vs herencia

Python + C++

Complete la tabla comparativa entre los mecanismos de abstracción de Python y C++, y entre composición y herencia como estrategias de diseño.

Aspecto	Python (ABC / Protocol)	C++17 (clase abstracta pura)
¿Puede instanciarse directamente?		
¿Puede tener estado (atributos)?		
¿Puede tener implementaciones?		
¿Cómo se declara método abstracto?		
¿Requiere herencia explícita?		
¿Herencia múltiple de interfaces?		

Criterio de diseño	Composición	Herencia de implementación
Flexibilidad en tiempo de ejecución		
Facilidad para pruebas unitarias		
Reutilización de código		
Acoplamiento resultante		
¿Cuándo usar?		

ACTIVIDAD 5 — Proyecto Integrador — Refactorizar con interfaces y composición

Python + C++

Revise el código del Proyecto Integrador de las Semanas 1-3 y aplique las siguientes mejoras:

- Defina al menos 1 interfaz (ABC) que represente un CONTRATO importante del dominio (ej: IAlmacenable, IExportable, INotificable, ICalculable). La interfaz debe tener al menos 2 métodos abstractos.

- Identifique una clase del proyecto que actualmente HAS-A varias responsabilidades distintas. Extraiga al menos una responsabilidad como interfaz e impleméntela por composición en lugar de por herencia o inline.
- Demuestre inyección de dependencias: al menos una clase debe recibir su dependencia (la interfaz) en el constructor en lugar de crearla internamente.
- Actualice el diagrama UML con la notación correcta para interfaces (<<interface>>) y composición (rombo relleno ◆). Exporte como PNG e inserte en el notebook.
- Commit: "refactor(semana4): interfaces, composicion e inyeccion de dependencias"

Justificación de diseño (completar):

Interfaz definida:	
Métodos abstractos:	
Clase que la implementa: **Clase que la CONSUME (composición):**	
¿Por qué composición y no herencia aquí?	

VI. PREGUNTAS DE REFLEXIÓN

1. En Python, la clase Protocol permite que un objeto "implemente" una interfaz sin declarar herencia explícita (duck typing estructural). ¿Cuál es la ventaja de Protocol frente a ABC? ¿Cuándo usaría uno y cuándo el otro?
2. El código que crea sus propias dependencias internamente (self._db = BaseDatos()) viola el principio de Inversión de Dependencias. ¿Qué problema práctico surge al querer probar unitariamente ese código? ¿Cómo la inyección de dependencias lo resuelve?
3. C++ permite herencia múltiple de clases con implementación (no solo interfaces). ¿Qué problemas puede causar esto? ¿Por qué Java y Python recomiendan herencia múltiple solo de interfaces/ABCs y no de clases concretas?

VII. RÚBRICA DE EVALUACIÓN

Criterio	Excelente (4)	Bueno (3)	Regular (2)	Insuf. (1)	Peso
Sistema notificaciones Python (Act. 1)	ABC; implementaciones correctas; sistema alertas; verificaciones.	Ejecutable con errores menores.	ABC definida sin implementaciones.	No ejecuta.	**20%**
Composición reportes + análisis (Act. 2)	Ejecutable; composición correcta; 3 preguntas respondidas.	Ejecutable; 1-2 preguntas.	Código parcial.	No entrega.	**20%**
Sistema notificaciones C++ (Act. 3)	Compila -Wall; interfaz pura; unique_ptr; SistemaAlertas.	Compila con warnings menores.	Compila sin unique_ptr.	No compila.	**20%**
Tablas comparativas (Act. 4)	Ambas tablas completas;	1 tabla completa.	Tablas parciales.	No entrega.	**15%**

	criterios técnicos correctos.				
Proyecto integrador (Act. 5)	Interfaz definida; composición+DI implementada; UML actualizado; commit.	Interfaz sin DI; UML sin actualizar.	Solo interfaz; sin composición.	No entrega.	**25%**

VIII. REFERENCIAS

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Martin, R. C. (2017). Clean Architecture. Pearson. https://www.pearson.com/en-us/subject-catalog/p/clean-architecture/P200000009528

Larman, C. (2004). Applying UML and Patterns (3rd ed.). Prentice Hall.

Python Software Foundation. (2026). abc — Abstract Base Classes. https://docs.python.org/3/library/abc.html

cppreference.com. (2026). Abstract class. https://en.cppreference.com/w/cpp/language/abstract_class